Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Fairlane - A Fair and Fault-Tolerant Scheduler for Distributed ML Training Workloads

Student / Team Information

Team Name: *Team #*	*Orchestrators* *OSDBMS-V-2026-T065*
Team member 1 (Team Lead)	*Sagar, Vansh – 240211442* *240211442@geu.ac.in* 
Team member 2	*Bhatnagar, Suryansh – 24021380* *24021380@geu.ac.in* 
Team member 3	*Sharma, Harshvardhan – 240211112* *240211112@geu.ac.in* 
Team member 4	*Garg, Vansh – 24021044* *24021044@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

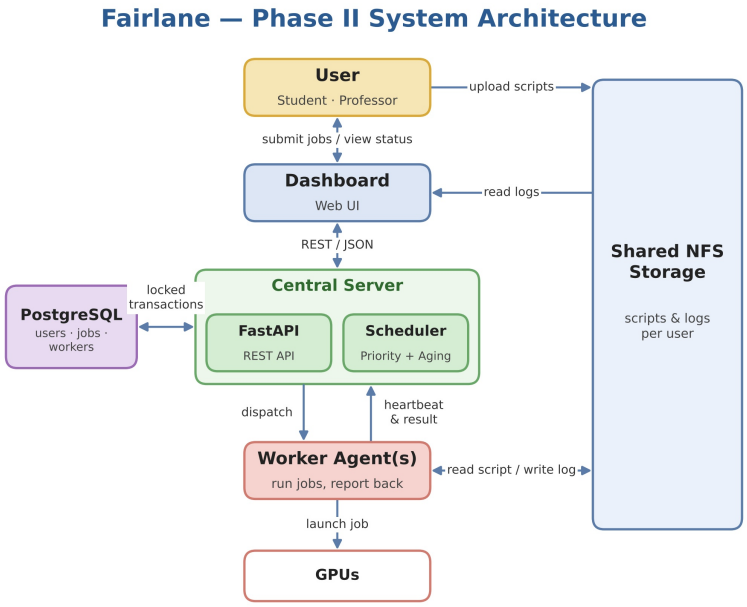
Project Abstract

Sharing a handful of GPUs among students can become messy. One person may wait long or someone else may take all the resources. Enterprise tools such as Slurm and Run:ai can solve this problem but they are too heavy for a laboratory. Therefore we created Fairlane. A scheduler for small GPU clusters.

The process is very simple. Users upload their scripts and submit jobs using a web dashboard. A central server holds the queue chooses the job and assigns it to a free worker. The worker executes the job. Sends the status back. For scheduling we chose priority with aging. The a job waits the higher its priority becomes so nobody remains stuck, in the queue forever. To keep track of everything we used PostgreSQL. The key part was using row-level locks and transactions so that two workers do not pick the job by mistake. If a worker stops during a task the system simply returns the job to the queue.

Honestly the best part was watching how operating system scheduling and database concurrency control fit together in a project. It is no longer theory.

Updated Project Approach and Architecture (2 pts)



Fairlane uses PostgreSQL 16 SQLAlchemy 2.0 and Ubuntu 24.04 with uv managed environments. The database schema that includes users, jobs, workers, executions and a race-free FCFS scheduler core is done. It uses SELECT... FOR UPDATE SKIP LOCKED to make sure there is no assignment when many people are working at the same time. Next steps are priority with aging GPU memory checks and Alembic migrations.

The NFS storage layer is done. A central export is mounted the way on all nodes. Relative paths make sure it can move easily. Server installation and documentation are complete. What is left are -node mount validation, permission hardening and checking container read and write access.

The Worker Agent starts training jobs on specific GPUs using CUDA_VISIBLE_DEVICES. It uses NVML based -flight checks for memory and usage. GPU isolation and running jobs on the same GPU are tested. The last step is connecting with the schedulers dispatch and completion contract.

The FastAPI layer and dashboard are in line with the shared state enums and field definitions. This makes sure the API surface is the same as the scheduler and worker contracts. This way everything works together smoothly from start, to finish.

Tasks Completed

Task Completed	Team Member
• Set up PostgreSQL 16 on Ubuntu 24.04 with uv/Python 3.12 and connection scripts. • Designed normalized schema covering users, jobs, and workers. • Implemented FCFS scheduling logic.	Vansh Sagar
• Established shared NFS server and created export directory across machines. • Enforced per-user file isolation (root_squash, UID matching, 750 permissions). • Authored setup documentation and shared folder conventions with team.	Harshvardhan Sharma
• Implemented API layer for job submission and status checks. • Provided a user interface/system for monitoring job states. • Established project-wide visual identity.	Vansh Garg
• Built functional Worker Agent for receiving dispatches and executing jobs. • Verified remote execution of Python training scripts on targeted GPUs. • Confirmed GPU/CPU isolation using CUDA_VISIBLE_DEVICES.	Suryansh Bhatnagar

Challenges/Roadblocks

Python environment lockout (PEP 668). Ubuntu 24.04 blocks pip install. Python environment lockout (PEP 668) was solved by switching to uv, which gives every member an environment that is pinned by a lock file and does not clash with the operating system package manager. Solved.

Worker state machine didn't fit one column. A node can be ONLINE and DEGRADED at the time for example when one GPU is dead and three GPUs are healthy. Worker state machine (didn't fit one column) was fixed by splitting the state into two parts: a status column that holds ONLINE, DRAINING or OFFLINE and a separate boolean flag called is_degraded. This change produces queries and removes any ambiguity.

Non-deterministic FCFS. Jobs that were submitted within the second returned in random order, which broke fairness. Non-deterministic FCFS was resolved by adding job_id as a tie-breaker: the job with the oldest timestamp wins and if timestamps are equal the job with the lowest ID wins. This fix was verified in tests.

NFS loopback mount conflict. Using /mnt/shared_jobs as both the server's export path and the client mount point causes a problem when the server also acts as a worker. NFS loopback mount conflict was fixed by exporting /srv/fairlane from the server while all machines, including the server, mount that export at /mnt/shared_jobs.

Multi-GPU modeling (open). The current database schema assumes that each worker has one GPU but real hardware boxes contain four to eight GPUs. Multi-GPU modeling (open) requires a gpus child table for each worker storing index, VRAM and health information so that the scheduler can perform per-GPU memory fit checks and pack multiple jobs onto a single card. The design of this feature is scheduled for the phase.

Multi-node NFS testing (open). Everything functions on a single machine but we still require at least three additional machines connected to the same network switch along with firewall rules, persistent /etc/fstab entries and synchronized UID/GID values across all hosts in order to validate the compute_workers group and the 750 permission model from start to finish. Lab access for this work is still pending.

NFS trust model (accepted). Plain NFS trusts the UIDs reported by clients. In a network this can be a vulnerability but, within our trusted laboratory environment it is an accepted trade-off. NFS trust model (accepted) is documented in the project assumptions.

Tasks Pending

Task Pending	Team Member (to complete the task)
• Complete schema extensions for executions, queues, and logs. • Implement Priority-with-Aging scheduling logic. • Guarantee single-worker job assignment via row-level locks. • Agree with Suryansh on dispatch and callback data contracts.	Vansh Sagar
• Test storing shared Python virtual environment directly on NFS. • Author repeatable setup guide for adding new machines to cluster. • Share finalized NFS path conventions with Vansh Garg and Vansh Sagar.	Harshvardhan Sharma
• Enhance UI/UX design for better visual appeal and usability. • Align payload fields (submission, status, worker registration) with Vansh Sagar. • Align script path input formatting with Harshvardhan's NFS layout. • Synchronize system state names (QUEUED, RUNNING, ONLINE) across UI.	Vansh Garg
• Validate multi-job-per-GPU memory sharing with simultaneous runs. • Implement live NVML usage/VRAM pre-flight checks before assignment. • Finalize dispatch and completion-callback contract with Vansh Sagar.	Suryansh Bhatnagar

Project Outcome/Deliverables (2 pts)

1. **A working prototype system** that fairly schedules and runs ML training jobs across multiple worker nodes, not just a design on paper.
2. **Measurable results** showing fairness between users, good resource utilization, and clean recovery from simulated worker failures.
3. **An interactive dashboard** where users can submit jobs and track whether they are waiting, running, or completed.
4. **Practical applicability** beyond the classroom — the same core ideas could scale to a real shared-GPU lab environment.
5. **Room to grow**, with dynamic worker registration meaning new machines can join the pool without touching the scheduler's code.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Repository: `github.com/neongit938-GH/Distributed_GPU_Workload_Orchestrator`
Branch: `main` (member branches: `Vansh_Branch`, `Garg_Dashboard`)

Important commits:

- `b82da3f` – Project setup: uv environment (Python 3.12) and PostgreSQL connection check.
- `5a6219e` – Scheduler and database: SQLAlchemy models (users, jobs, workers), seed data, FCFS job selection with job_id tie-breaker.
- `37ac671` – NFS setup guide (README_NFS.md) and Phase II task sheet uploaded.
- `7df9647` – Phase II tasks divided among team members.
- `ad51110` – Merged scheduler code into `main`.

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	**Status (Pass/Fail)**	**Notes**
Database connectivity	Pass	SQLAlchemy connects to PostgreSQL 16 and reads the server version.
Schema creation	Pass	users, jobs and workers tables created with keys, foreign key and named CHECK constraints.
Constraint rejection	Pending	Manual psql test: insert an invalid status and an unknown user, both must be rejected.
FCFS job selection	Pass	Skips non-QUEUED jobs; equal timestamps resolved by lower job_id.
Worker eligibility filter	Pass	DRAINING and degraded workers correctly excluded.
Concurrent assignment (two schedulers)	Pending	Race test with FOR UPDATE SKIP LOCKED: no job may be assigned twice.
NFS server installation	Pass	nfs-kernel-server installed and service active.
NFS mount and permissions	Pending	root_squash, UID matching and 750 folders; single machine first, then multi-node.
Remote job execution on target GPU	Pass	Worker Agent runs Python training scripts on the assigned GPU.
GPU isolation	Pass	Verified with CUDA_VISIBLE_DEVICES.
Multi-job-per-GPU sharing	Pending	Simultaneous runs to check memory sharing on one GPU.
End-to-end dispatch to completion	Pending	Needs the scheduler–worker contract to be finalised.

Deliverables Progress (2 pts)

The working prototype is in progress. The scheduler core with its FCFS logic and row-level locking is done, the database schema is set, the NFS layer is configured, and the Worker Agent can launch jobs on specific GPUs with NVML checks. Each piece works on its own, so the focus now is wiring them together — getting the full dispatch-to-completion loop running end to end.

Measurable results on fairness, utilization, and failure recovery are waiting on that integration. We need the full system up on a multi-node testbed to run aging simulations, collect telemetry, and inject failures like killing a worker mid-job to verify clean re-queuing.

The dashboard is also in progress. The FastAPI endpoints for submission, status, and worker registration are written and match the shared contract, but real-time updates and role-based views still need work.

The design already supports a real shared-GPU lab — stateless scheduler, shared-nothing workers, NFS and PostgreSQL scaling horizontally. Dynamic registration is built in: the worker table, registration API, and heartbeat state machine (ONLINE/DRAINING) are in place. A new node just installs the agent, mounts NFS, and registers — no scheduler changes needed. The next milestone is validating the whole stack across multiple machines.